23. 動態規劃

2026資訊研究社 算法班
Made with ❤️ by jheanlee

動態規劃是什麼

動態規劃(dynamic programming) 的目標:

- 將一個複雜的問題切分成簡單的子問題
- 避免重複計算
- 優化子結構

簡單來說就是對程式問題賦予數學式的解法

動態規劃有兩種思考方式 :top-down 與 bottom-up

動態規劃實例—費氏數列 (top-down)

費氏數列的第 n 項為：第 n-1 項 + 第 n-2 項；而第 0 項定義為 0 、第 1 項定義為 1

透過遞迴方式我們可以快速地寫出解法：

```
int basic_fib(int n) {
  if (n <= 1) return n;
  return basic_fib(n - 1) + basic_fib(n - 2);
}
```

動態規劃實例—費氏數列 (top-down)

但如果我們將這個遞迴樹展開，我們會發現有許多步驟其實是重複計算的。

以 basic_fib(5) 為例 (basic_fib(n)省略為f(n))：

1. `f(5)`
2. `f(4) + f(3)`
3. `(f(3) + f(2)) + (f(2) + f(1))`
4. `(f(2) + f(1)) + (f(1) + f(0)) + (f(1) + f(0)) + f(1)`
5. `((f(1) + f(0)) + f(1)) + (f(1) + f(0)) + (f(1) + f(0)) + f(1)`

動態規劃實例—費氏數列 (top-down)

這時候我們可以使用一個表格幫我們記錄下已經處理過的資料。

這個技巧稱為memoisation

```
int memo_fib(int n, vector<int> &memo) {
  if (n <= 1) return n;
  if (memo[n] != -1) return memo[n];
  memo[n] = basic_fib(n - 1) + basic_fib(n - 2);
  return memo[n];
}
```

動態規劃實例—費氏數列 (bottom-up)

如果我們反過來思考：在多層遞迴後，我們建構出了一個 memo 表。那我們能不能直接將答案建構於表上而捨棄遞迴呢？

這便是 bottom-up 寫法的精神，我們透過一個陣列(可能多維)，將解答直接建構出來。

這個寫法是從第0 項開始向上建構，與遞迴相反。

```cpp
int bottom_up_fib(int n) {
  vector<int> dp(n + 1);

  //  初始條件
  dp[0] = 0;
  dp[1] = 1;

  //  開始迭代
  for (int i = 2; i <= n; i++) {
    dp[i] = dp[i - 1] + dp[i - 2];
  }

  return dp[n];
}
```

動態規劃實例—費氏數列 (bottom-up)

Bottom-up 寫法有兩個要點

1. 初始條件 (base case)
2. 一般項的定義

只要搞清楚這兩點便能輕鬆寫出動態規劃解法
(雖然要想到很難啦)

動態規劃實例—費氏數列 (bottom-up 降維)

有時我們直覺下所想到的解法並非最有效率的解法。必須在其中看出其他浪費掉的空間或時間並進行優化。

費氏數列計算時僅需要前兩項的資料即可，不需要所有項的資料。因此我們可以僅保留前兩項於變數中，將空間複雜度自O(n) 降為 O(1)。

```
int bottom_up_optimised_fib(int n) {
  if (n <= 1) return n;

  int prev = 0; int curr = 1;
  for (int i = 2; i <= n; i++) {
    int tmp = prev + curr;
    prev = curr;
    curr = tmp;
  }

  return curr;
}
```

Leetcode 70 - Climbing Stairs

這題是經典的爬樓梯問題。有一個n 階的樓梯，一步可以踩一階或跨兩階。在只往上走的狀況下，請問幾種到頂的方式？

Example 1:

> **Input:** n = 2
> **Output:** 2
> **Explanation:** There are two ways to climb to the top.
> 1. 1 step + 1 step
> 2. 2 steps

Example 2:

> **Input:** n = 3
> **Output:** 3
> **Explanation:** There are three ways to climb to the top.
> 1. 1 step + 1 step + 1 step
> 2. 1 step + 2 steps
> 3. 2 steps + 1 step

Leetcode 70 - Climbing Stairs (top-down)

```cpp
class Solution {
private:
  int climb(int n, vector<int> &memo) {
    if (memo[n] != -1) return memo[n];
    memo[n] = climb(n - 2, memo) + climb(n - 1, memo);
    return memo[n];
  }

public:
  int climbStairs(int n) {
    vector<int> memo(n + 1, -1);
    memo[0] = 1;
    memo[1] = 1;

    return climb(n, memo);
  }
};
```

Leetcode 70 - Climbing Stairs (bottom-up)

```cpp
class Solution {
public:
  int climbStairs(int n) {
    vector<int> dp(n + 1);
    dp[0] = 1;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
      dp[i] = dp[i - 1] + dp[i - 2];
    }

    return dp[n];
  }
};
```

Leetcode 70 - Climbing Stairs (bottom-up; OP)

```cpp
class Solution {
public:
  int climbStairs(int n) {
    int prev = 1, curr = 1;

    for (int i = 2; i <= n; i++) {
      int tmp = curr + prev;
      prev = curr;
      curr = tmp;
    }

    return curr;
  }
};
```

Leetcode 746 - Min Cost Climbing Stairs

與上一題不同，這一題的每一階都有不同的價格。有一個n 階的樓梯，從第 i 階可以花費 cost[i] 元踩一階或跨兩階。請問到頂的最低花費為何？ (即找出最優路徑之花費)

Example 1:

> **Input:** cost = [10,15,20]
> **Output:** 15
> **Explanation:** You will start at index 1.
> - Pay 15 and climb two steps to reach the top.
> The total cost is 15.

Example 2:

> **Input:** cost = [1,100,1,1,1,100,1,1,100,1]
> **Output:** 6
> **Explanation:** You will start at index 0.
> - Pay 1 and climb two steps to reach index 2.
> - Pay 1 and climb two steps to reach index 4.
> - Pay 1 and climb two steps to reach index 6.
> - Pay 1 and climb one step to reach index 7.
> - Pay 1 and climb two steps to reach index 9.
> - Pay 1 and climb one step to reach the top.
> The total cost is 6.

Leetcode 746 - Min Cost Climbing Stairs (top-down)

```cpp
class Solution {
private:
  int minCost(vector<int> &cost, vector<int> &memo, int step) {
    if (step == 0 || step == 1) {
      return cost[step];
    }
    if (memo[step] != -1) {
      return memo[step];
    }
    memo[step] = min(minCost(cost, memo, step - 1), minCost(cost, memo, step - 2)) + cost[step];
    return memo[step];
  }
public:
  int minCostClimbingStairs(vector<int>& cost) {
    vector<int> memo(cost.size(), -1);
    return min(minCost(cost, memo, cost.size() - 1), minCost(cost, memo, cost.size() - 2));
  }
};
```

Leetcode 746 - Min Cost Climbing Stairs (bottom-up)

```cpp
class Solution {
public:
  int minCostClimbingStairs(vector<int>& cost) {
    vector<int> dp(cost.size());
    dp[0] = cost[0];
    dp[1] = cost[1];

    for (int i = 2; i < cost.size(); i++) {
      dp[i] = min(dp[i - 1], dp[i - 2]) + cost[i];
    }

    return min(dp[cost.size() - 1], dp[cost.size() - 2]);
  }
};
```

Leetcode 72 - Edit Distance

DP 不只是一維單方向運行，也有二維、三維的。這題是二維DP的經典題。

給予兩個字串，目標是要以最小操作步數將word1 編輯成 word2

每一步驟可以:
1. 新增一個字
2. 刪除一個字
3. 取代一個字

Example 1:

```
Input: word1 = "horse", word2 = "ros"
Output: 3
Explanation:
horse -> rorse (replace 'h' with 'r')
rorse -> rose (remove 'r')
rose -> ros (remove 'e')
```

Example 2:

```
Input: word1 = "intention", word2 = "execution"
Output: 5
Explanation:
intention -> inention (remove 't')
inention -> enention (replace 'i' with 'e')
enention -> exention (replace 'n' with 'x')
exention -> exection (replace 'n' with 'c')
exection -> execution (insert 'u')
```

Leetcode 72 - Edit Distance

```
class Solution {
private:
  vector<vector<int>> traversed;
  int dp(string &w1, int i1, string &w2, int i2) {
    if (i1 == -1) return i2 + 1;    //  if w1 ends, add the rest of w2's characters
    if (i2 == -1) return i1 + 1;    //  if w2 ends, delete the rest of w1's characters

    if (traversed[i1][i2] != -1) return traversed[i1][i2];

    if (w1[i1] == w2[i2]) {
      traversed[i1][i2] = dp( [&] w1, i1 - 1,  [&] w2, i2 - 1); //  same character, pass
    } else {
      traversed[i1][i2] = min(
          dp( [&] w1, i1 - 1,  [&] w2, i2) + 1, //  delete one character from w1
           b: min(
              dp( [&] w1, i1,  [&] w2, i2 - 1) + 1, //  add one character to w1
              dp( [&] w1, i1 - 1,  [&] w2, i2 - 1) + 1  //  replace one character of w1
          )
      );
    }

    return traversed[i1][i2];
  }
public:
  int minDistance(string word1, string word2) {
    traversed = vector<vector<int>> ( n: word1.size(),  value: vector<int>( n: word2.size(),  value: -1));
    return dp( w1:  [&] word1,  i1: word1.size() - 1,  w2:  [&] word2,  i2: word2.size() - 1);
  }
};
```

Leetcode 72 - Edit Distance

先處理無腦的狀況

```
int dp(string& w1, int i1, string& w2, int i2) {
    if (i1 == -1) return i2 + 1;    //  if w1 ends, add the rest of w2's characters
    if (i2 == -1) return i1 + 1;    //  if w2 ends, delete the rest of w1's characters
```

w1 = app
w2 = apple

app -> appl -> apple

w1 = apple
w2 = app

apple -> appl -> app

Leetcode 72 - Edit Distance

定義遞迴函式`int dp(string &w1, int i1, string &w2, int i2)` 回傳 `w1[0..=i1]` 與 `w2[0..i2=]` 的編輯距離

每個位置可以做3種動作(或者相同字的話就跳過):

新增

a	p	p	l	s

a	p	p	l	e	s

刪除

a	p	p	l	e	s

a	p	p	l	e

取代

a	p	p	l	a	s

a	p	p	l	e	s

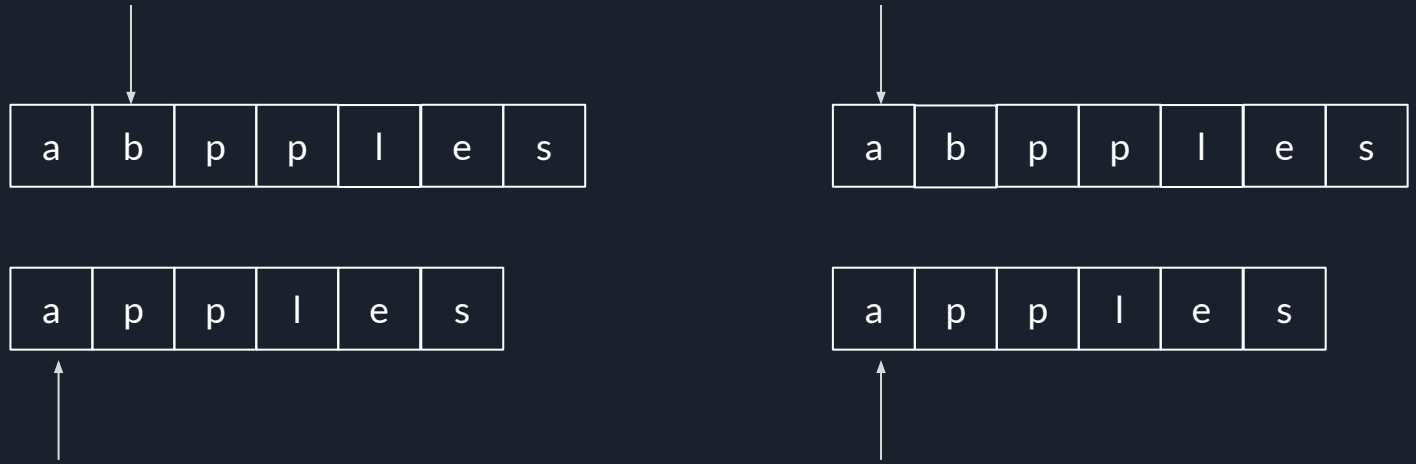
Leetcode 72 - Edit Distance

新增


```
dp( [&] w1, i1, [&] w2, i2 - 1) + 1,
```

Leetcode 72 - Edit Distance

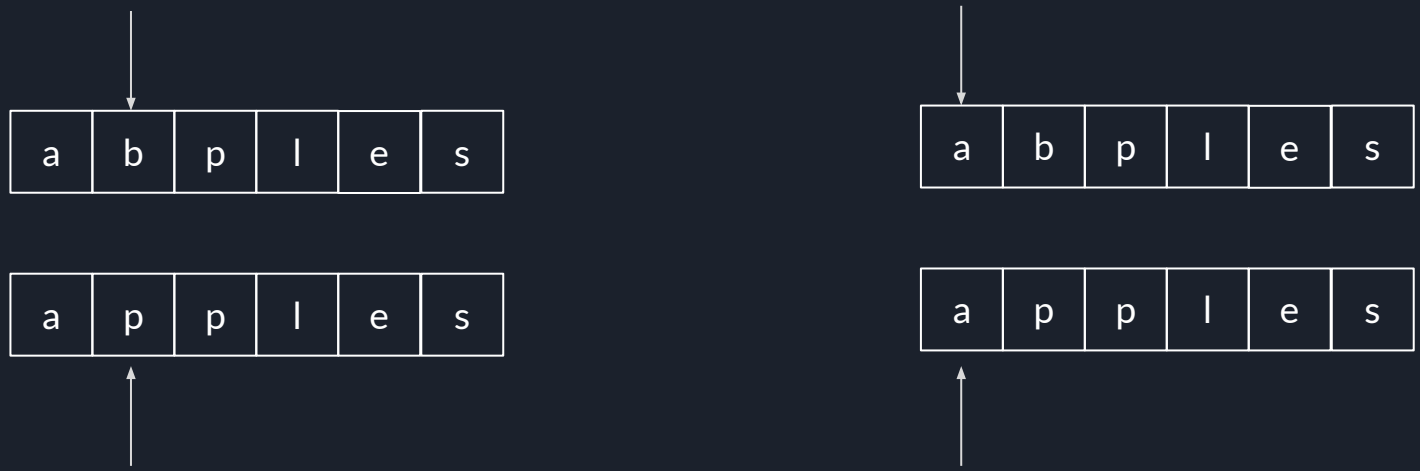
删除




```
dp([&]w1, i1 - 1, [&]w2, i2) + 1,
```

Leetcode 72 - Edit Distance

取代




```
dp([&]w1, i1 - 1, [&]w2, i2 - 1) + 1
```

Leetcode 72 - Edit Distance

最後再把搜尋過的memo 起來

```
vector<vector<int>> traversed;
int dp(string &w1, int i1, string &w2, int i2) {
  if (i1 == -1) return i2 + 1;    //  if w1 ends, add the rest of w2's characters
  if (i2 == -1) return i1 + 1;    //  if w2 ends, delete the rest of w1's characters

  if (traversed[i1][i2] != -1) return traversed[i1][i2];

  if (w1[i1] == w2[i2]) {
    traversed[i1][i2] = dp( [&] w1, i1 - 1,  [&] w2, i2 - 1); //  same character, pass
  } else {
    traversed[i1][i2] = min(
        dp( [&] w1, i1 - 1,  [&] w2, i2) + 1, //  delete one character from w1
         b: min(
            dp( [&] w1, i1,  [&] w2, i2 - 1) + 1, //  add one character to w1
            dp( [&] w1, i1 - 1,  [&] w2, i2 - 1) + 1  //  replace one character of w1
        )
    );
  }

  return traversed[i1][i2];
}
```